More Machine Learning & Wrapup

John Guttag

MIT Department of Electrical Engineering and Computer Science

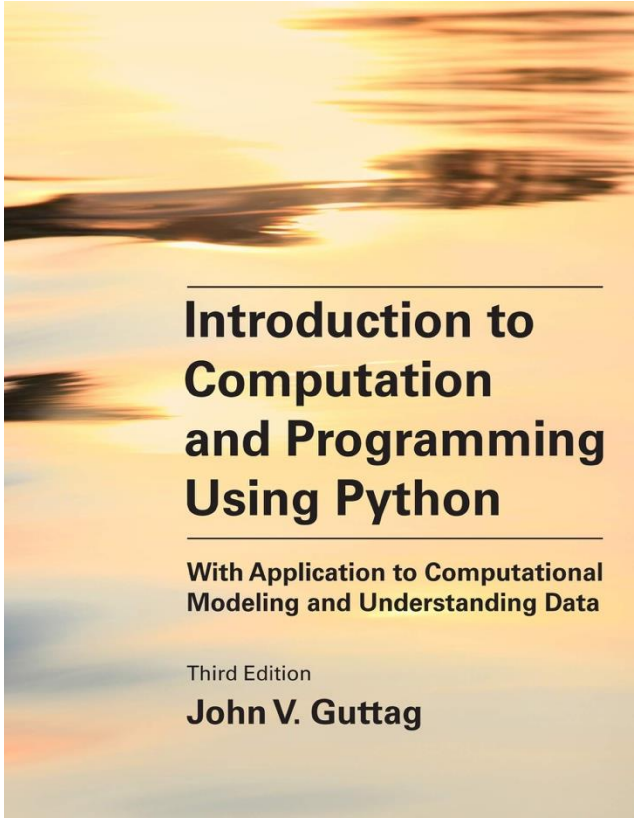
Announcements

- Today is the last lecture

- April 29, last quiz
- May 6, problem set 5 due

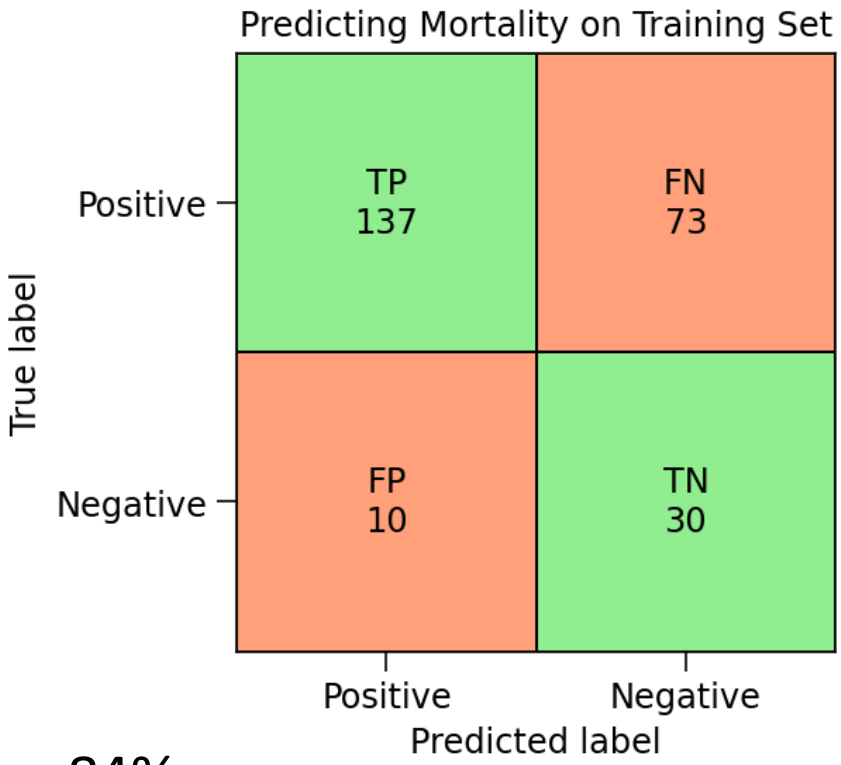
Assigned Reading

- Today: Chapter 25
- Next lecture: NaN



Left Off Talking About Decisions Trees

```
|--- numAttacks <= 0.50
|   |--- stElev <= 0.50
|   |   |--- class: 0.0
|   |--- stElev >  0.50
|   |   |--- class: 1.0
|--- numAttacks >  0.50
|   |--- heartRate <= 54.00
|   |   |--- class: 1.0
|   |--- heartRate >  54.00
|   |   |--- class: 1.0
```



Mean accuracy on training set = 84%

Mean recall on training set = 88%

Mean precision on training set = 71%

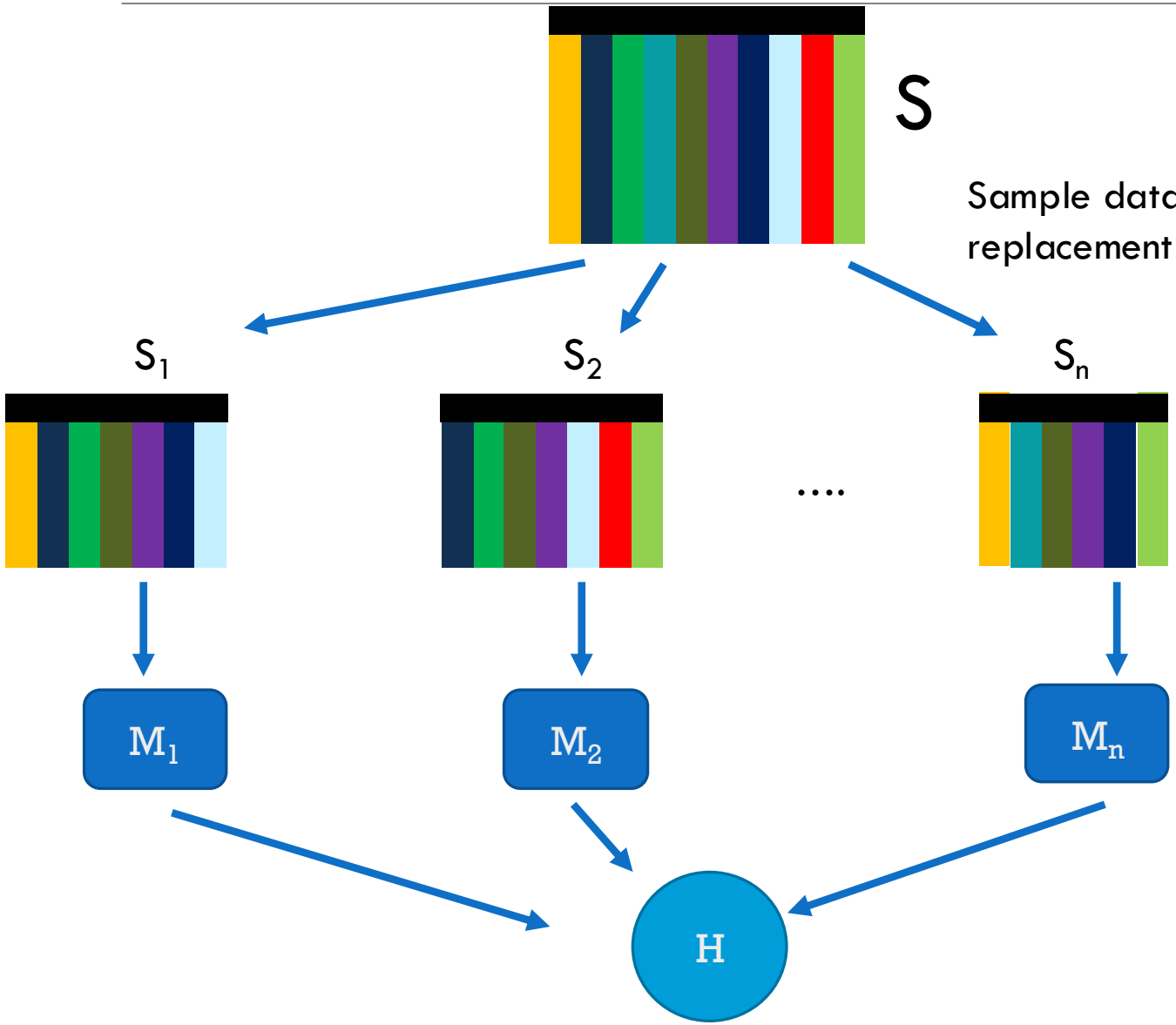
If 2 is better than 1, why stop there?

But Tree of Depth 12 Overfit to Training Data

Tree of Depth 12	Tree of Depth 12
Mean accuracy on training set = 90%	Mean accuracy on test set = 80%
Mean recall on training set = 88%	Mean recall on test set = 68%
Mean precision on training set = 84%	Mean precision on test set = 68%

- Algorithm finds good decision tree for training data
- But it can overfit to training data, so that performance on training data is misleading
- One solution: random forests
  - Train multiple trees, each on randomly chosen subset of training data and random subset of features
  - At inference time, make a prediction using each tree
  - Combine predictions, e.g., by taking a vote

Random Forrest



Sample data with replacement

Avoid correlation across trees

Select random subset of features to consider at each node of the decision tree

Build classifiers over samples

Use majority Vote for classification (or avg. for regression)

Implementing Random Forests

- Not hard to code in basic Python, but why would we?

```
from sklearn.ensemble import RandomForestClassifier
```

```
RF = RandomForestClassifier(
    n_estimators=num_trees,
    max_depth=max_depth,
    max_features = 'sqrt',
    bootstrap = True)
model = RF.fit(X_train, Y_train)
```

Define the kind of model
Fixed number of trees
Upper bound on tree depth
Number of features considered for each split
Sample with replacement
Fit model to data

```
y_hats = model.predict(X)
```

Let's Try It on Cardiac Data

- Perform 100 trials of
  - One decision tree of depth 8
  - Random forest of 100 decision trees of depth 8

100 trials of 1 trees with max depth 8	100 trials of 100 trees with max depth 8
Mean accuracy on training = 90%	Mean accuracy on training = 99%
Mean recall on training = 85%	Mean recall on training = 99%
Mean precision on training = 86%	Mean precision on training = 99%
Mean accuracy on test = 77%	Mean accuracy on test = 82%
Mean recall on test = 63%	Mean recall on test = 76%
Mean precision on test = 66%	Mean precision on test = 72%

How Does It Compare to Simple Baselines?

100 trials of 100 trees with max depth 8
Mean accuracy on test = 82%
Mean recall on test = 76%
Mean precision on test = 72%

For 100 trials of majority class baseline
Mean accuracy on test = 67%
Mean recall on test = 0%
No positive predictions

For 100 trials of weighted random baseline
Mean accuracy on test = 56%
Mean recall on test = 32%
Mean precision on test = 33%

Ensemble Methods

- Random forests an example of an ensemble method
- Basic idea is to find a useful way to combine a set of weak (but better than random) classifiers to get a strong classifier
- Works best when the weak classifiers make different mistakes
- Goal is to
  - Reduce variance, less sensitive to noise in training data
  - Reduce bias, avoid systematic errors
  - Improve generalization

Two Common Approaches

- Bootstrap aggregation
  - Train independent models on distinct subsets of training data
  - E.g., random forests
- Boosting (e.g., AdaBoost, XGBoost)
  - Train models sequentially
  - Model n+1 focuses on correcting the mistakes of model n
    - By increasing weight on examples misclassified by earlier models
  - Final result based on a weighted average of all the models

AdaBoost

1. Start by giving each example the same weight, e.g., 1/n

2. Train a simple classifier, e.g., a shallow decision tree, with goal of minimizing its **weighted** error rate

3. Assign to that model an importance based on its weighted error rate

5. Update weights of examples, giving more weight to the examples the previous model got wrong

6. Repeat steps 2-5 the desired number of times

7. Final prediction is an importance-weighted majority vote

AdaBoost vs Random Trees (one example only)

100 trials of AdaBoost with 4-level decision tree
Mean accuracy on training = 100%
Mean recall on training = 99%
Mean precision on training = 100%
Mean accuracy on test = 79%
Mean recall on test = 66%
Mean precision on test = 68%

100 trials of 100 trees with max depth 4
Mean accuracy on training = 92%
Mean recall on training = 90%
Mean precision on training = 85%
Mean accuracy on test = 83%
Mean recall on test = 80%
Mean precision on test = 72%

What We Have Covered So Far

- Machine learning, the big picture
- Evaluating classifier
- Decision tree classifiers
- Idea of an Ensemble classifier
- Random Forests

Sometimes We Want Probabilities

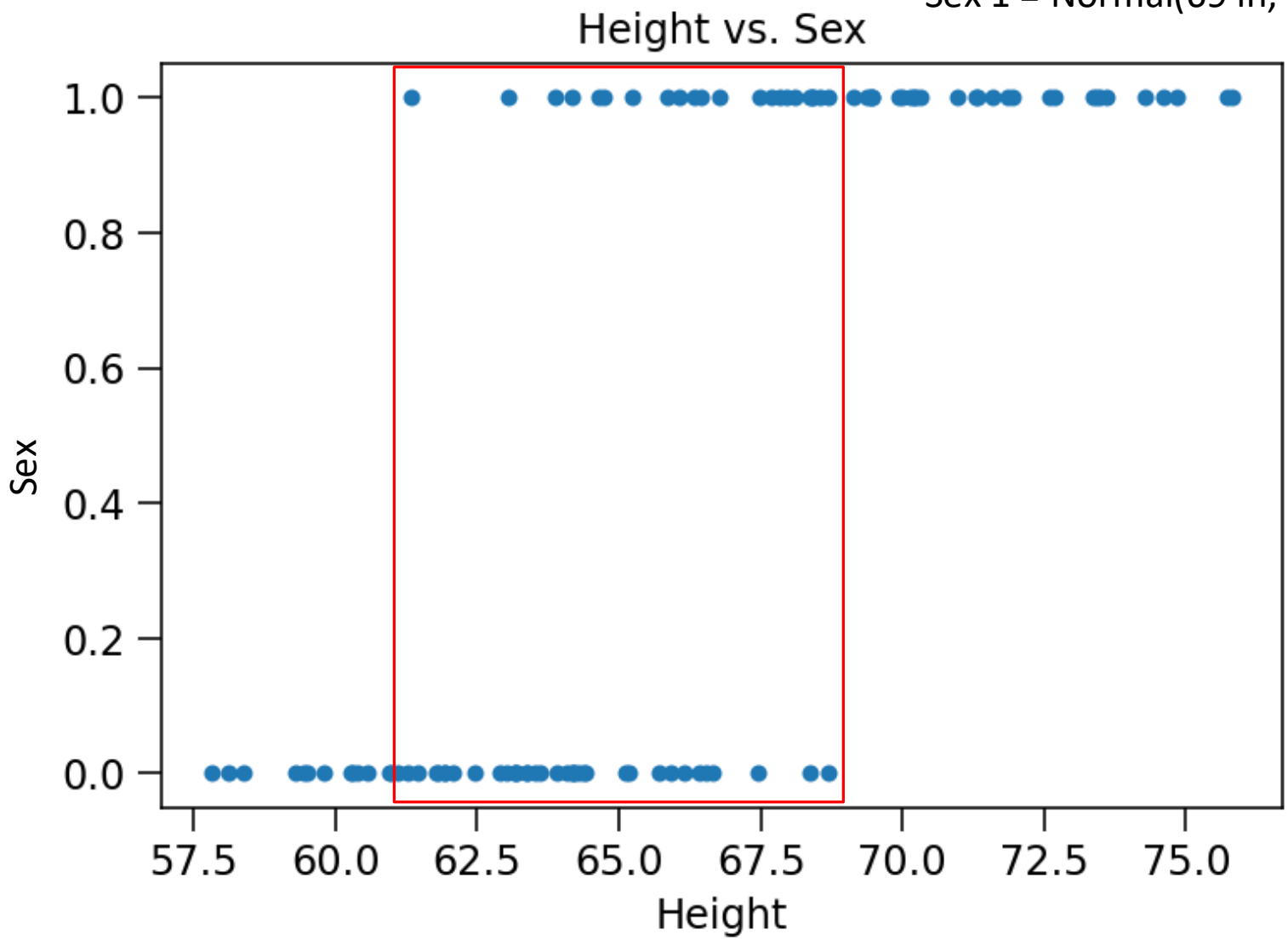
- So far, classifiers have produced a binary value
- But sometimes we want a probability
  - Can change decision threshold to manage trade off between recall and precision
  - Can help us understand how confident we should be in prediction
- Use regression to get a probability
  - Use a threshold to turn probability into a predicted class
- How about using linear (polynomial) regression?

Predicting Birth Sex from Height

Heights sampled from two sexes

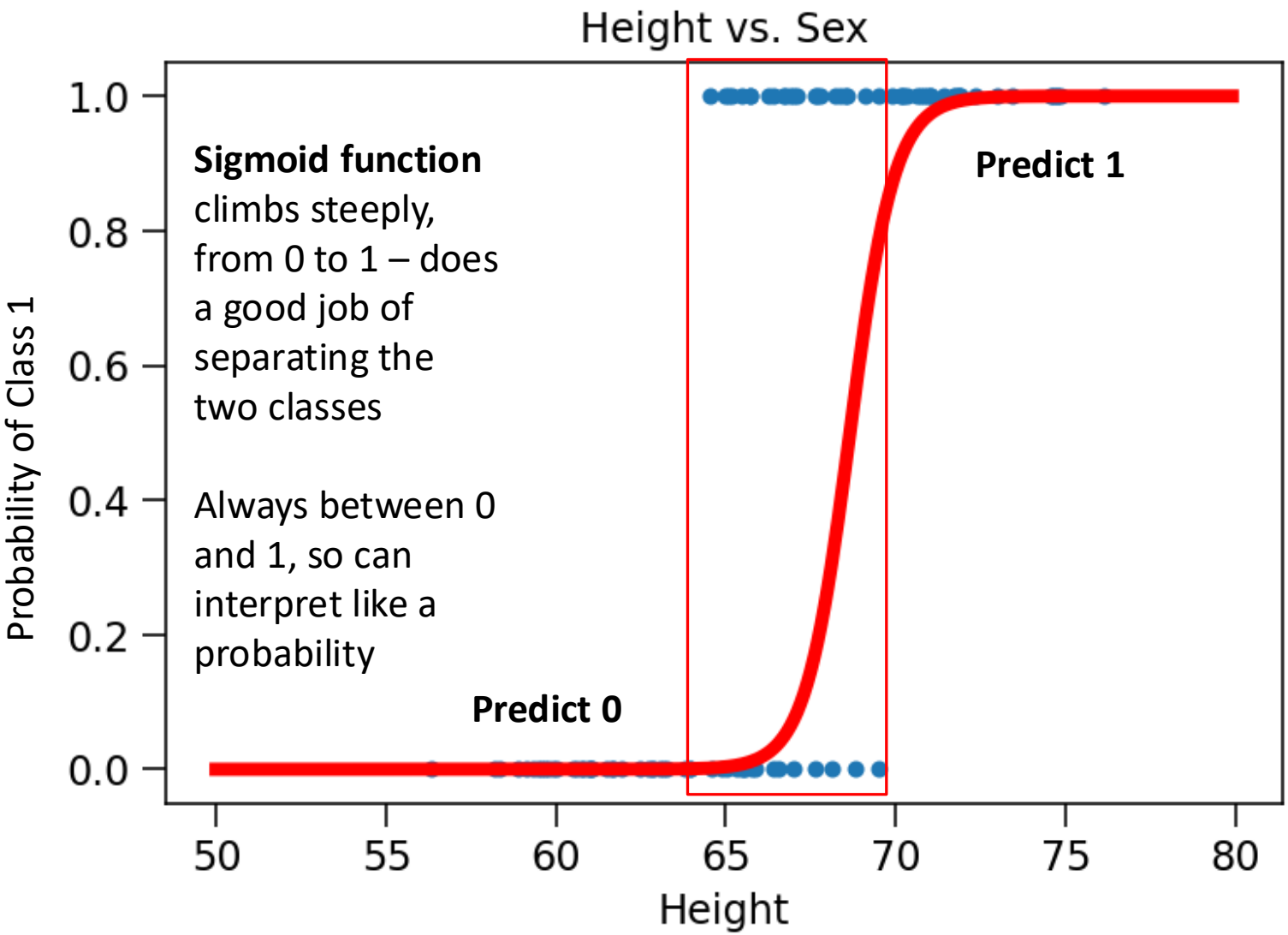
Sex 0 = Normal(63 in, 3 in)

Sex 1 = Normal(69 in, 3 in)



Try Linear Regression

Idea: Regress with A Different Function



Sigmoid function climbs steeply, from 0 to 1 – does a good job of separating the two classes

Always between 0 and 1, so can interpret like a probability

Logistic Regression

General form of sigmoid is $\sigma(z) = \frac{1}{1+ e^{-z}}$ Never < 0, never > 1

Suppose we have *d* dimensional data

Learn **weights** $\theta_0 \cdots \theta_d$ one for each feature plus an intercept θ_0

Intercept analogous to b in a line: $y = ax + b$

For an example x with features x_1 .. x_d with label y, we can compute:

$$x_\theta = \theta_0 + \theta_1 x_1 \cdots + \theta_d x_d$$

Plugging into the sigmoid function yields

$$\sigma(x_\theta) = \frac{1}{1 + e^{-x_\theta}}$$

Probability of label y = 1 is $\sigma(x_\theta)$

Logistic Regression Classifiers

- Perform classification via regression with a sigmoid
- Given training data with binary labels, fit a sigmoid to it
- Given a data point x, this gives us probability label = 1

$$\Pr(y = 1|x) = \sigma(x_\theta)$$

- Example: for our heights data, the best fitting sigmoid is

θ_0 = -85.31

θ_1 = 1.27

Can be extended to multi-class labeling, e.g., more than 2 sexes

$$pr(sex = 1) = \frac{1}{1 + e^{-\theta_0 + \theta_1 x_1}}$$

$$x_1 = height$$

Logistic Regression sklearn Package

- `LogisticRegression` is an sklearn class
- `LogisticRegression(max_iter, class_weight)`
  - creates an instance of class
- `LogisticRegression().fit(SetOfFeatures, setOfLabels)`
  - finds weights such that logistic function of linear combination of weights and features optimizes goodness of fit
- Result is an object containing coefficients (or weights) and function that will compute probability of label for any other feature vector

Let's Try It on Cardiac Data

Try 100 trials of a balanced logistic regression
Mean accuracy on training = 84%
Mean recall on training = 87%
Mean precision on training = 71%
Mean accuracy on test = 83%
Mean recall on test = 85%
Mean precision on test = 70%

Mean coefficients: HR = 0.002, stElev = 1.807
Age = 0.011, prevAttacks = 2.196

Provides some insight into relative importance of features
- Each positively correlated with outcome
- PrevAttacks and stElev most important

Birds-eye View of Machine Learning

Classification	Clustering, Association
fit a model, e.g., (image→ bird species), where prediction is from a discrete set of classes)	divide examples into groups based on their similarity E.g., split customers by their purchasing habits to do targeted marketing
Regression	**Other**
fit a model, e.g., (Weight → Spring Displacement), and use that function to predict displacement for other weights	(e.g., LLMs, dimensionality reduction, GANs,…)

Clustering

- **Cluster**: a collection of examples that share a set of similar properties
  - Ideally properties are numeric or can be converted to a numeric scale, so that clusters defined by sets of "nearby" examples
- Potential goals of clustering
  - Data reduction: extract one representative per cluster for further processing/analysis
  - Find "natural families" and study their properties
  - Find unusual data points (outlier detection)

Clustering as Optimization Problem

Minimize global inertia

$$inertia = \sum_{e \in c} distance(mean(c), e)^2$$

$$global_inertia = \sum_{c \in C} inertia(c)$$

c is a cluster

C is a collection of clusters

e is an example

Ducks

Cluster Ducks

Optimization Function

$$inertia = \sum_{e \in c} distance(mean(c), e)^2$$

$$global_inertia = \sum_{c \in C} inertia(c)$$

c is a cluster

C is a collection of clusters

e is an example

- Is optimization problem simply finding a C that minimizes *global_inertia(C)*?
  - No, otherwise could put each example in its own cluster

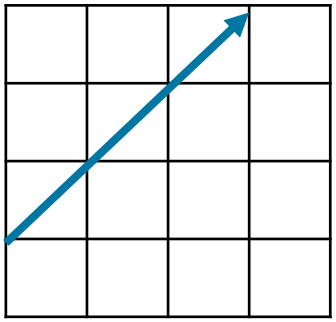
Need a constraint, for example:

Minimum distance between clusters

Number of clusters (maximum overall, or specific number)

Measuring Distance Between Features: Euclidean

$$\text{dist}(X1, X2) = \sqrt{\sum_{k=1}^{\text{len}} (X1_k - X2_k)^2}$$



Measuring Distance Between Features: Minkowski Metric

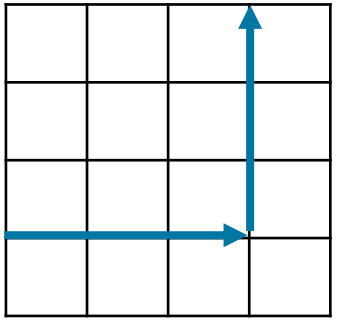
$$\text{dist}(X1, X2) = \sqrt[p]{(X1_k - X2_k)^p}$$

"L1 Norm" **p = 1: Manhattan Distance**

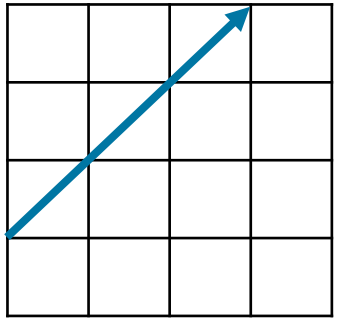
"L2 Norm" **p = 2: Euclidean Distance**

$$dist([0,1],[3,4],1) = abs(0-3) + abs(1-4) = 6$$

$$dist([0,1],[3,4],2) = \sqrt{(0-3)^2 + (1-4)^2} = 4.24$$



p=1



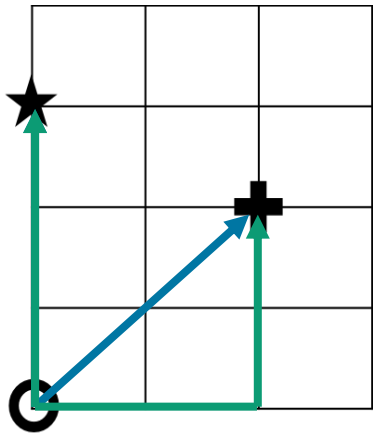
p=2

Typically use Euclidean metric

Choice of Metric Matters

$$dist(X1, X2, p) = \left(\sum_{k=1}^{len} abs(X1_k - X2_k)^p \right)^{1/p}$$

p = 1: Manhattan Distance
p = 2: Euclidean Distance



Is circle closer to star or cross?

- Euclidean distance
  - Cross – 2.8
  - Star – 3
- Manhattan Distance
  - Cross – 4
  - Star - 3

Inertia Example

$$inertia = \sum_{e \in c} distance(mean(c), e)^2$$

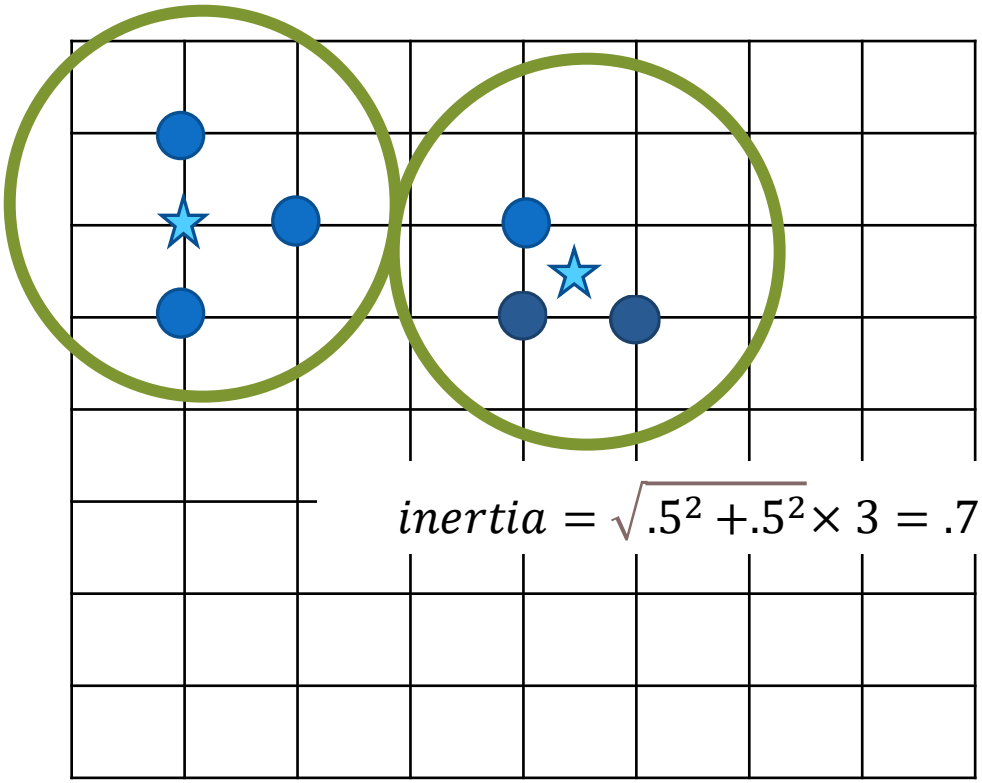
$$global_inertia = \sum_{c \in C} inertia(c)$$

Using Euclidian distance, find k =2 clusters that minimize global_inertia

Example

Mean (Centroid)

$$inertia = 1^2 \times 3 = 3$$



$$inertia = \sqrt{.5^2 + .5^2} \times 3 = .7 \times 3 = 2.1$$

$$global_inertia = 5.1$$

Inertia Example

$$inertia = \sum_{e \in c} distance(mean(c), e)^2$$

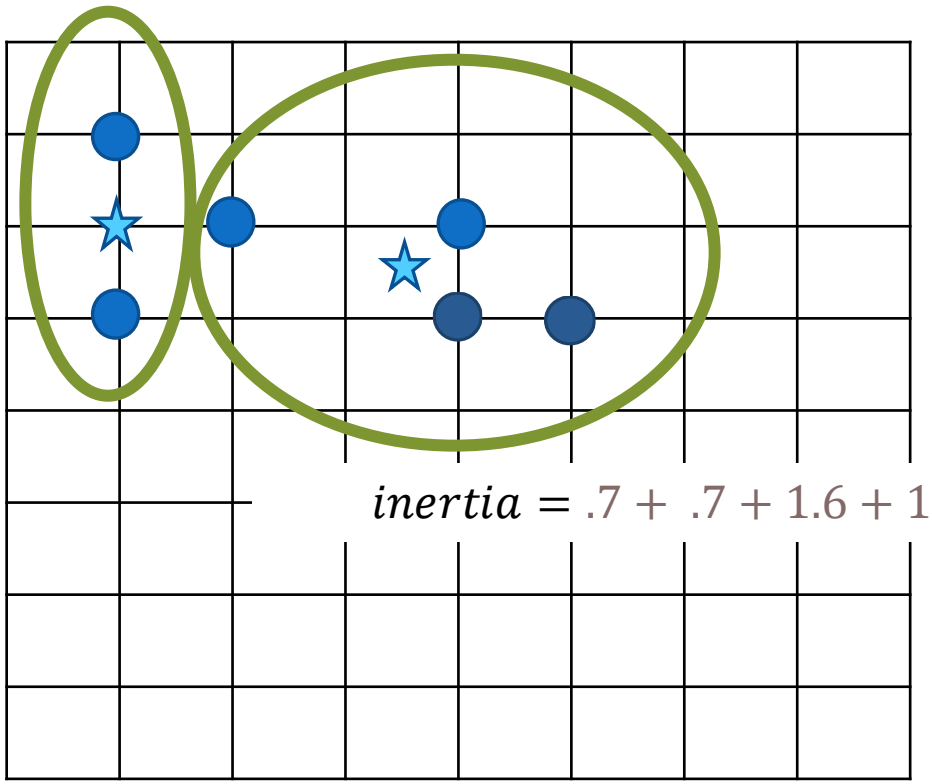
$$global_inertia = \sum_{c \in C} inertia(c)$$

Using Euclidian distance

Example

Mean (Centroid)

$$inertia = 1^2 \times 2 = 2$$

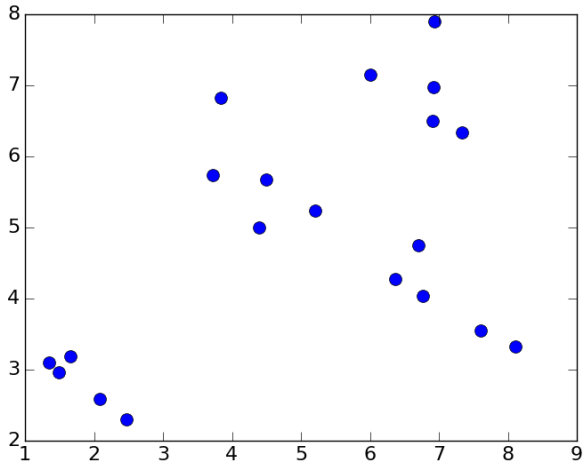


$$inertia = .7 + .7 + 1.6 + 1.6 = 4.6$$

$$global_inertia = 6.6$$

K-means Algorithm

- Exactly minimizing global inertia requires exponential time
- K-mean finds an approximation
- Chose a set of centroids at random
- Alternate between
  - Assigning each point to nearest centroid
  - Recomputing centroid as average of cluster



K-means Algorithm

```
randomly chose k examples as initial centroids
while true:
   create k clusters by assigning each
       example to closest centroid
   compute k new centroids by averaging
       examples in each cluster
   if centroids don’t change:
       break
```

K-means Algorithm

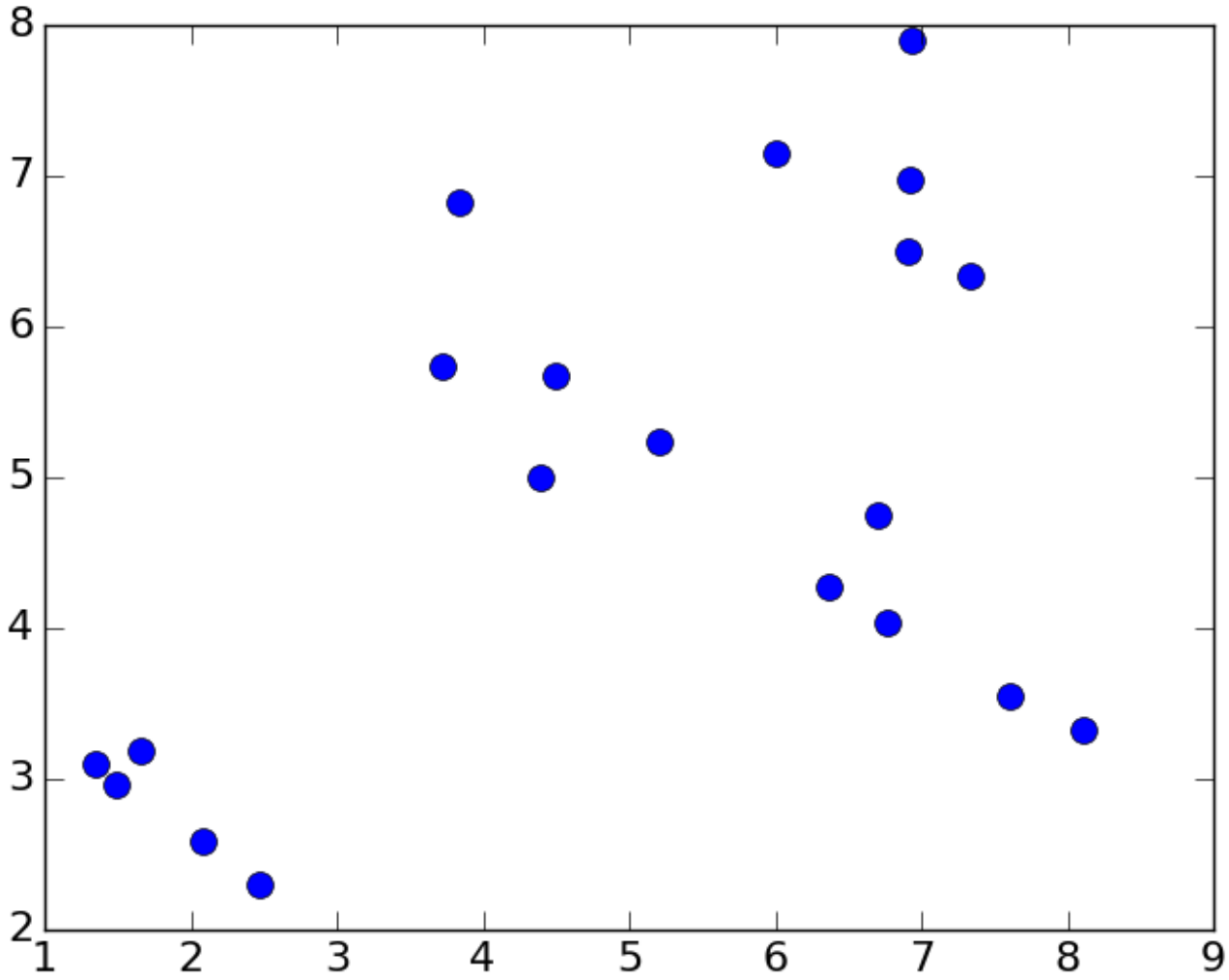
```
randomly chose k examples as initial centroids
while true:
   create k clusters by assigning each
       example to closest centroid
   compute k new centroids by averaging
       examples in each cluster
   if centroids don't change:
       break
```

What is complexity of one iteration?

k*n*d, where n is number of points and d time required to compute the distance between a pair of points

It's fast

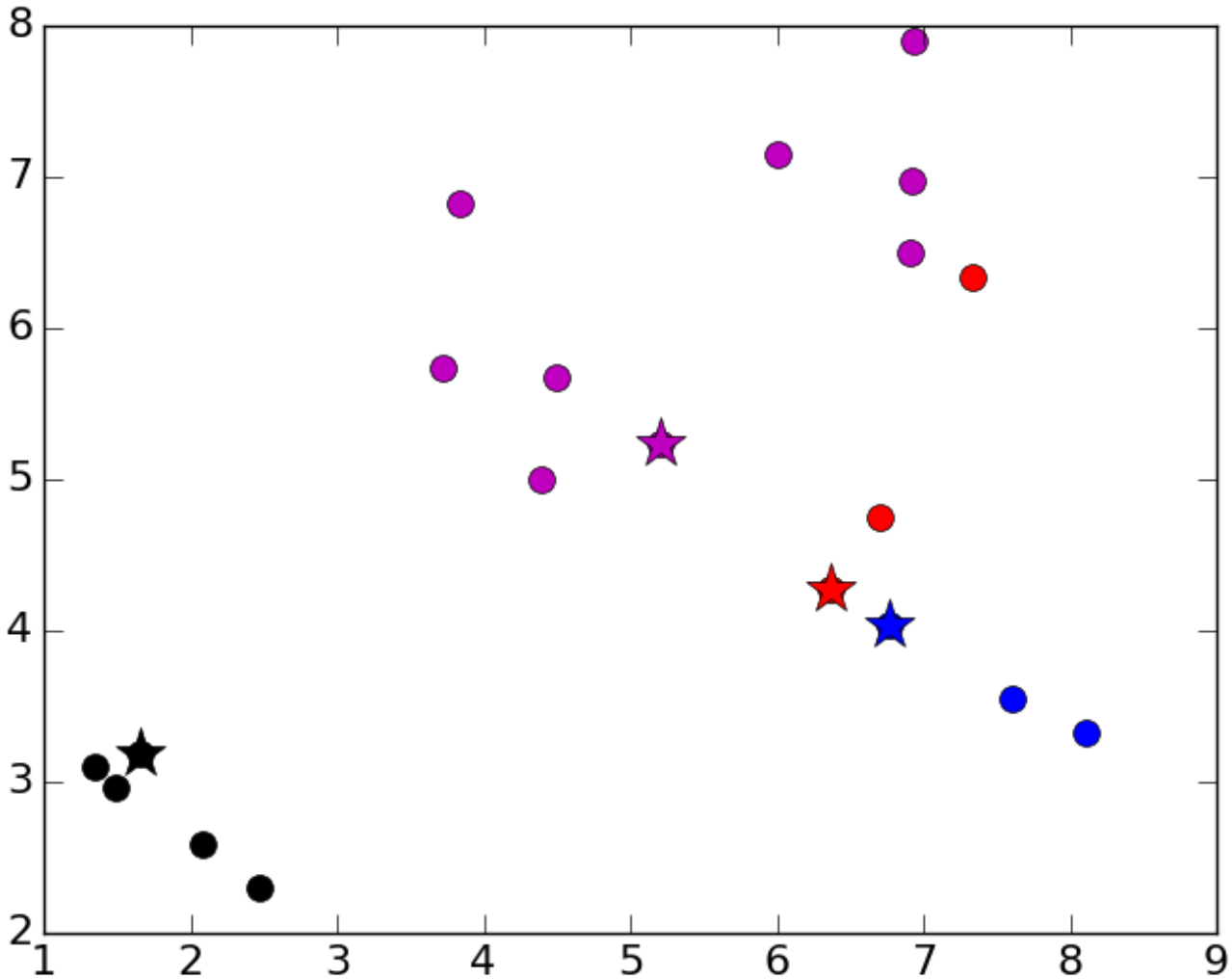
An Example



K = 4, Initial Centroids

Choose centroids at random;

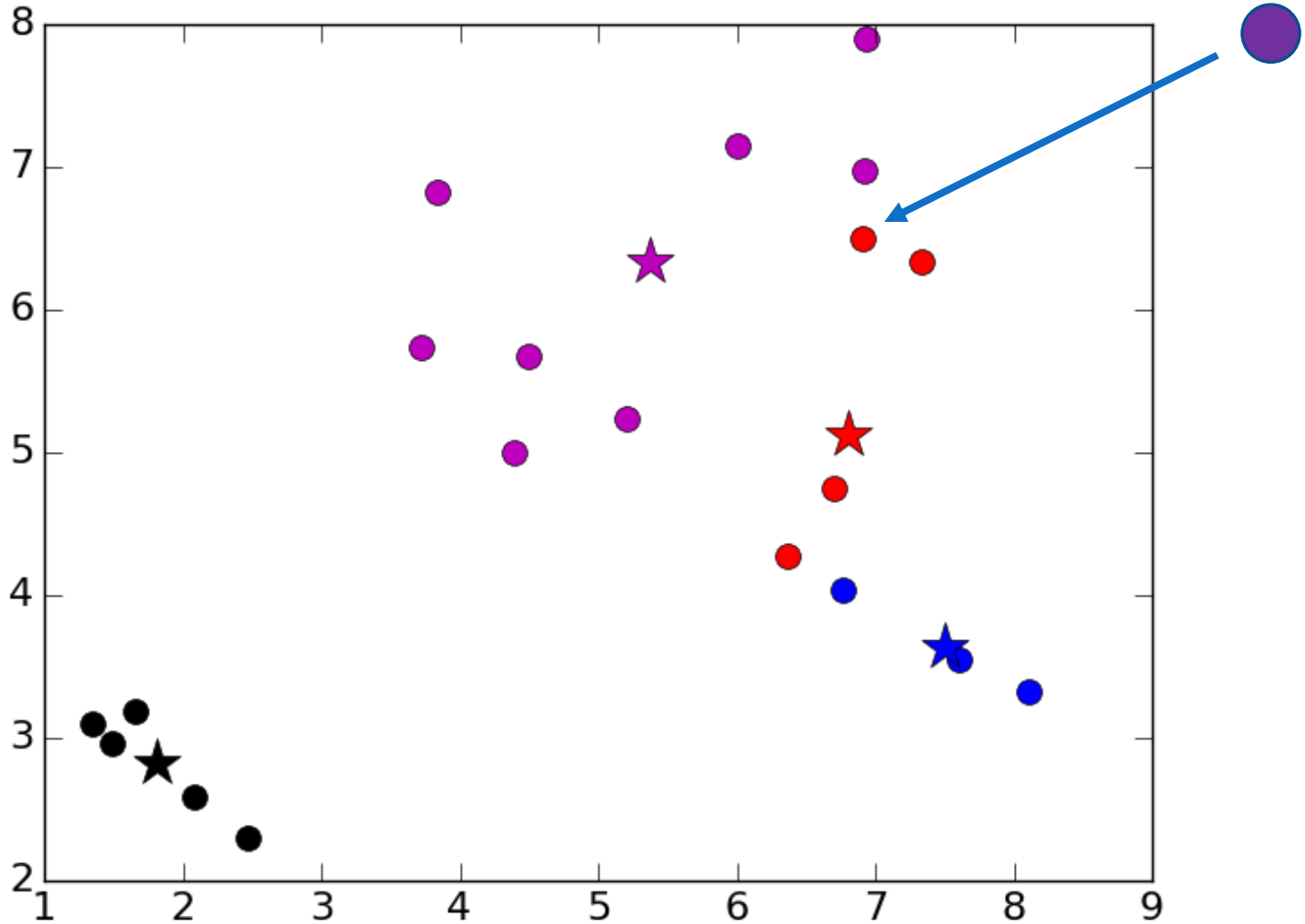
Assign each example to nearest centroid



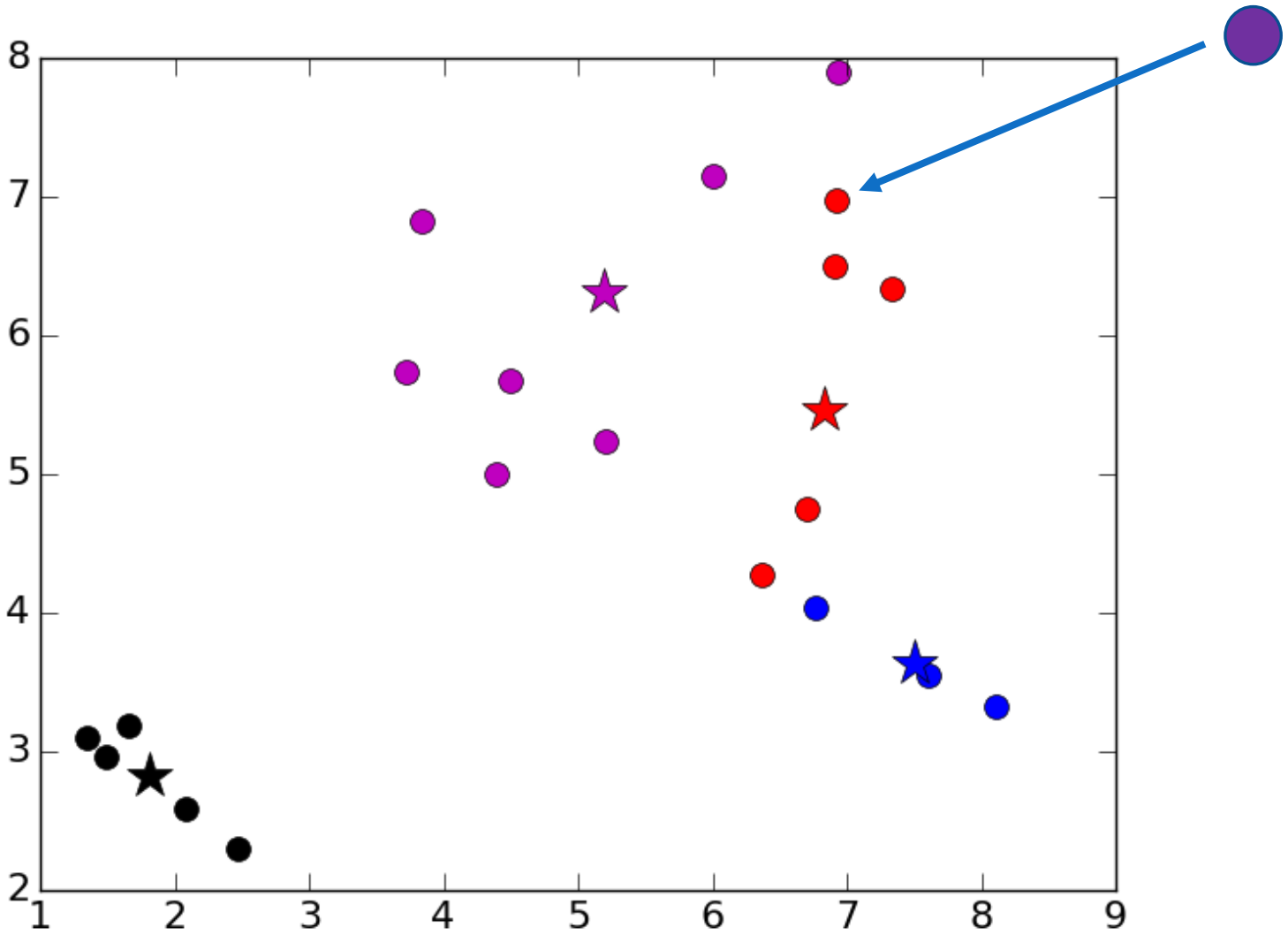
Iteration 1

Compute new centroids – will not correspond to examples

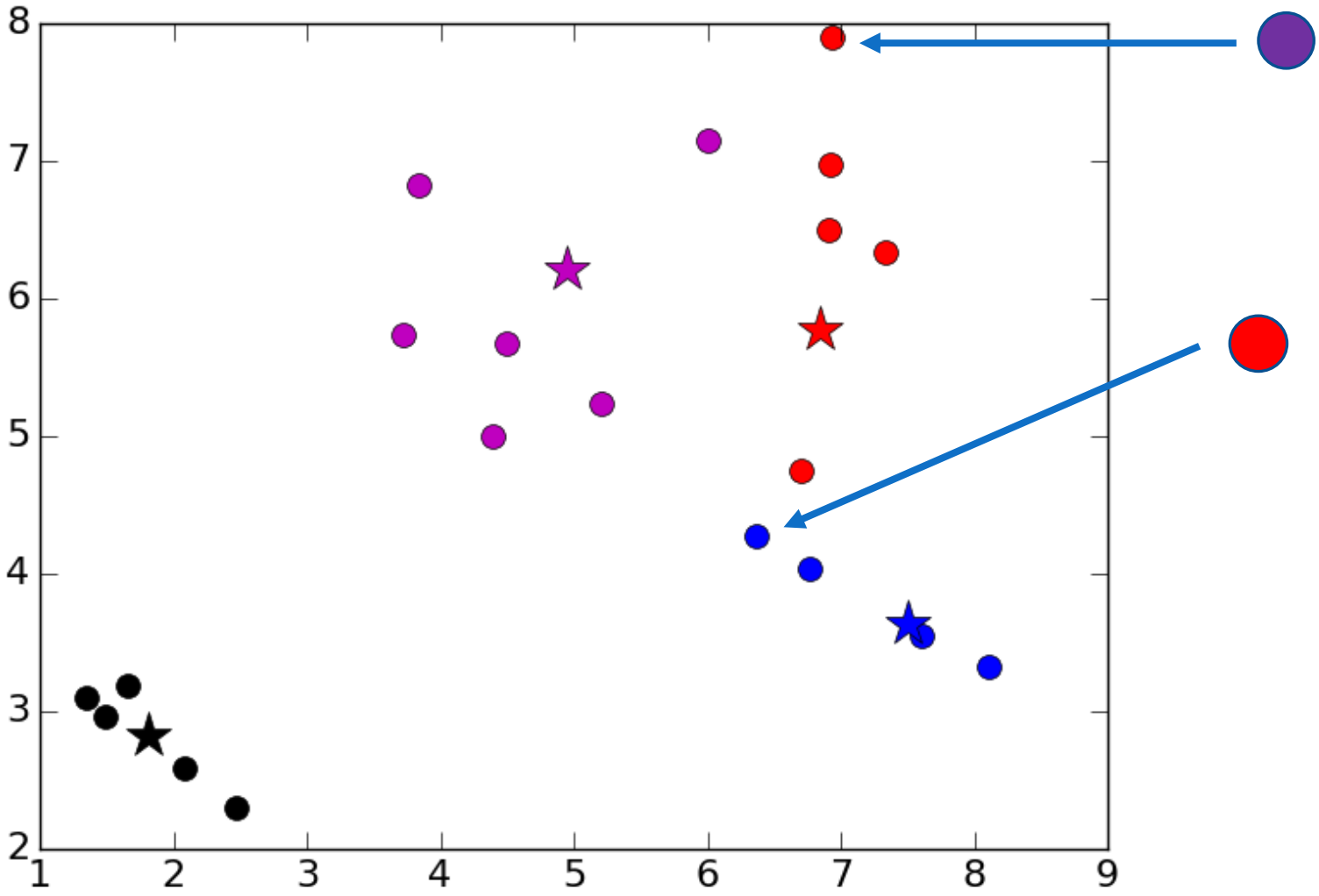
Assign each example to nearest centroid, note switch in label



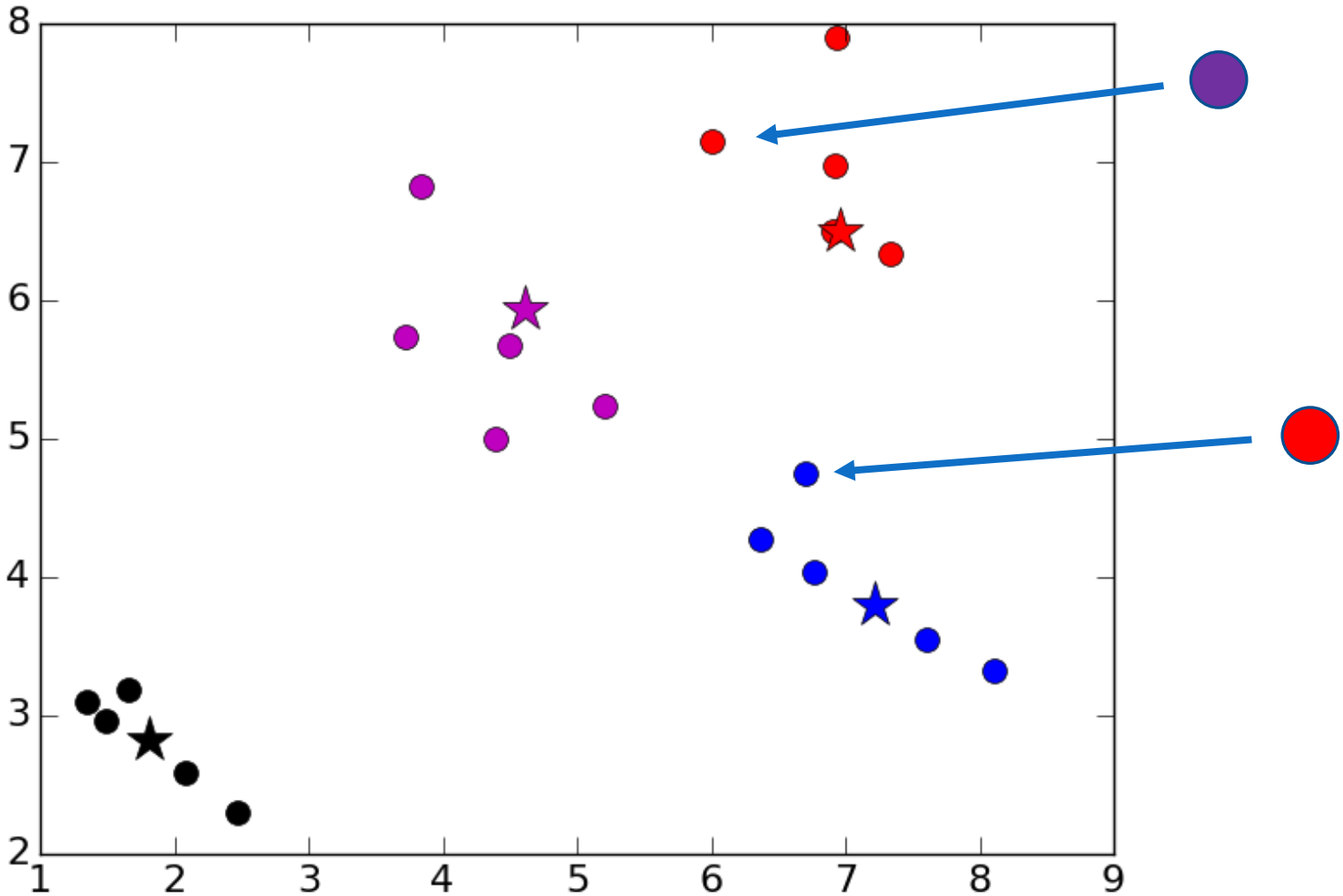
Iteration 2



Iteration 3

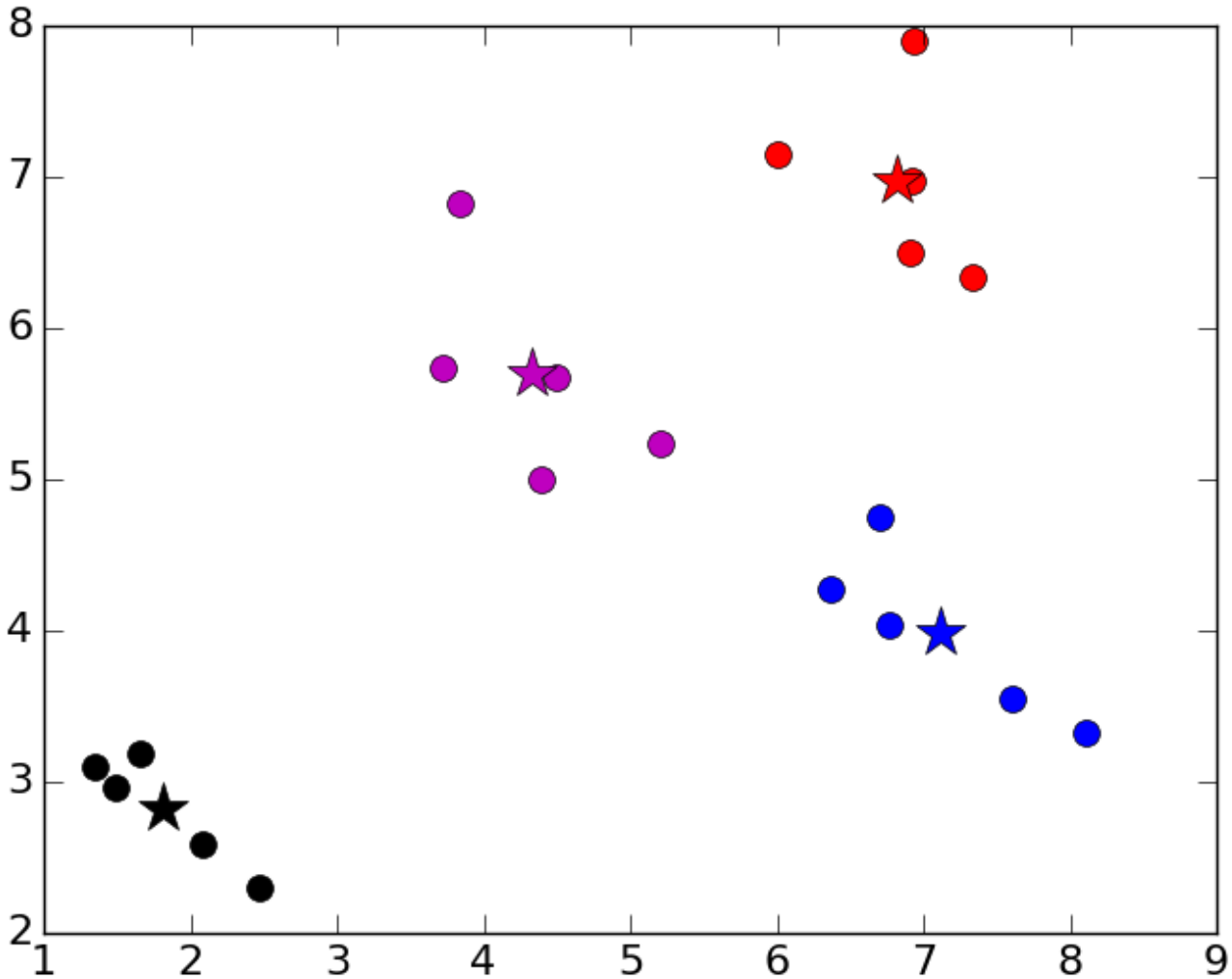


Iteration 4



Iteration 5

Centroids move, but no examples switch – can halt

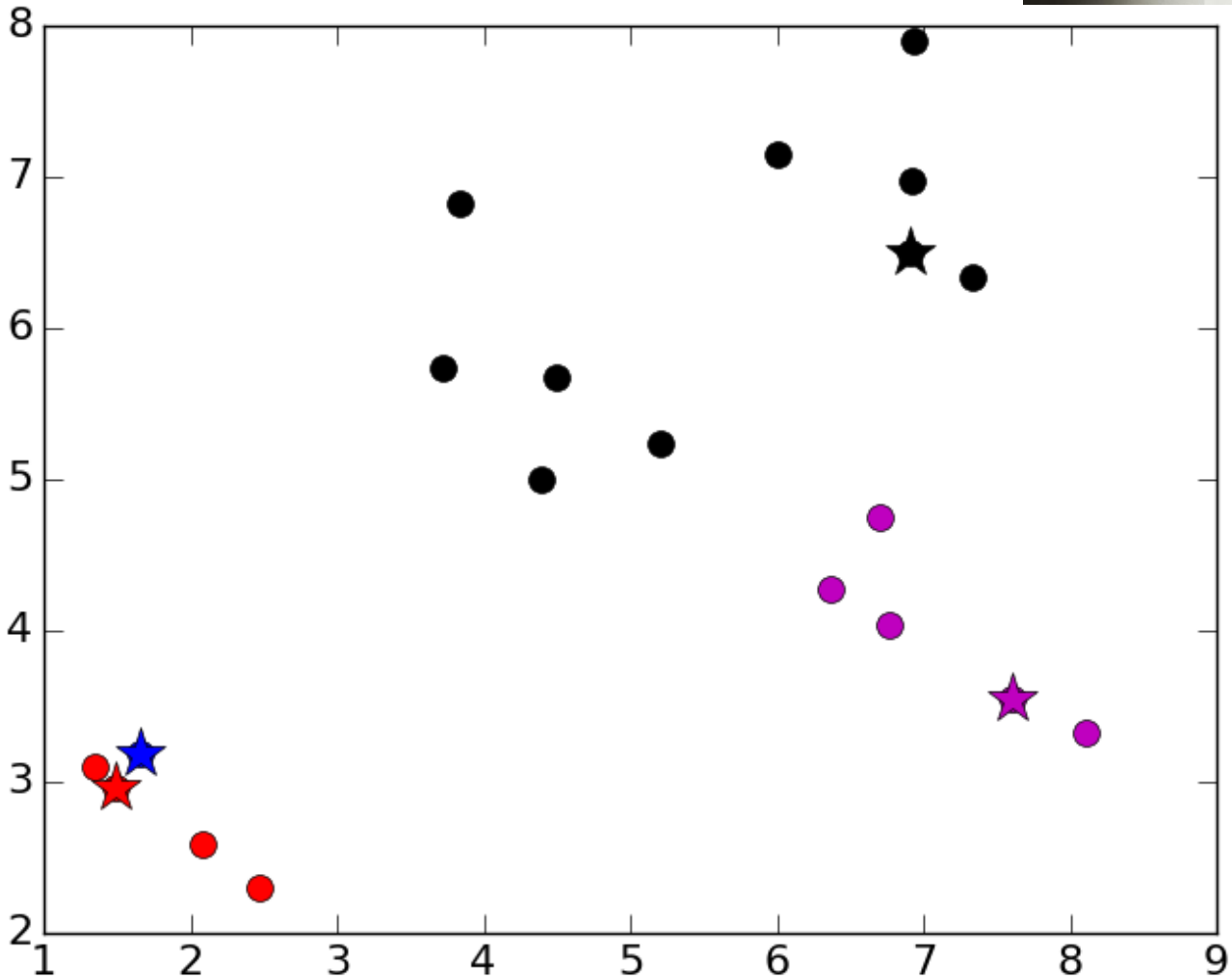


Questions

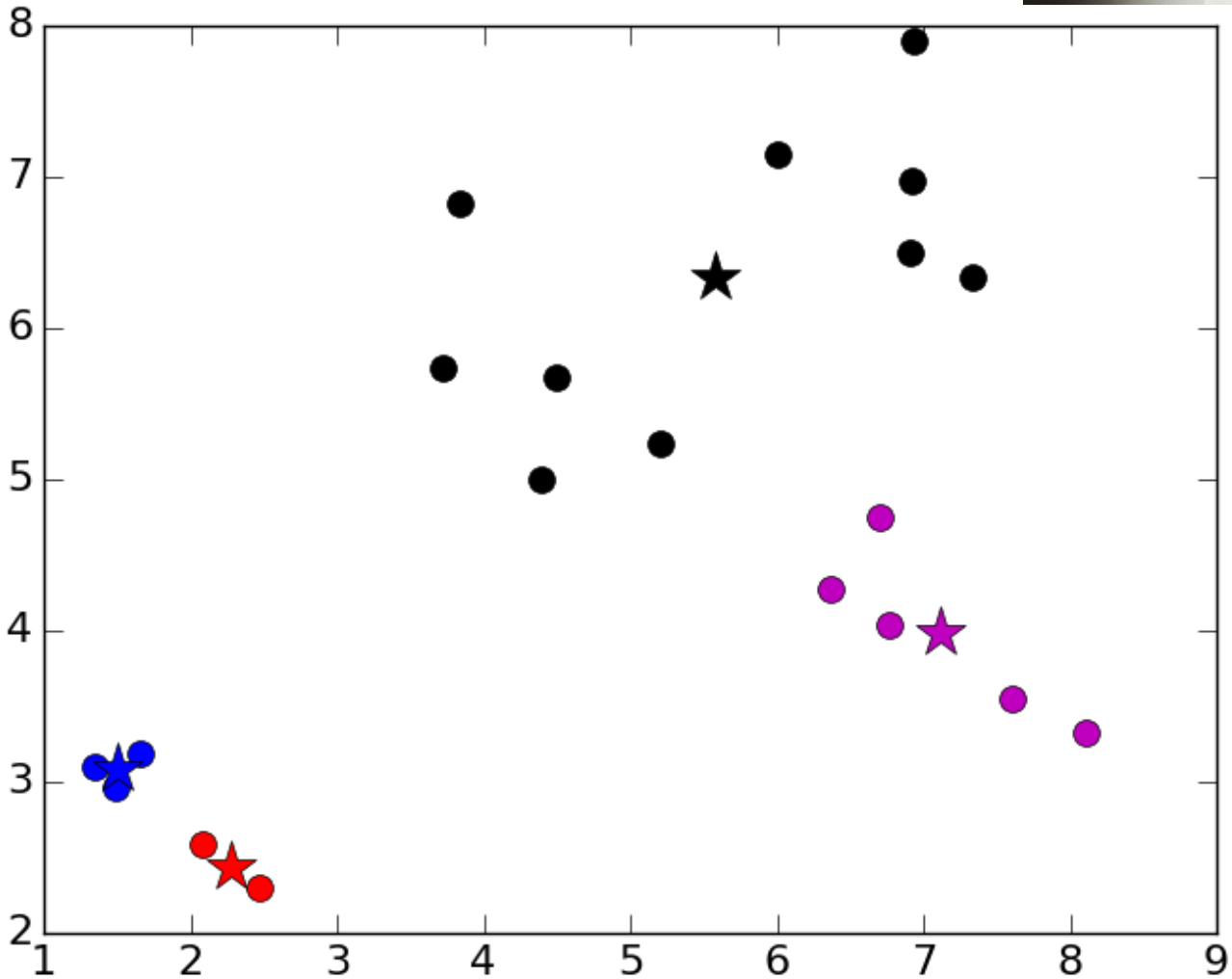
Which of the following are true?

1) Final clustering is independent of initial centroids False
2) Process guaranteed to converge True
3) Number of iterations independent of initial centroids False
4) Always finds the lowest inertia assignment given k False

Unlucky Initial Centroids



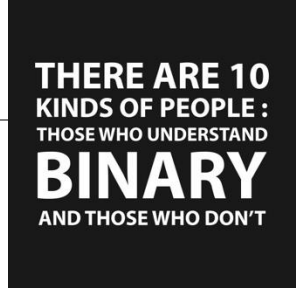
Converges On



Issues with k-Means

- Greedy algorithm can find different local optima
  - Result can depend upon
    - Choice of initial centroids
    - Number of iterations
- K is an important parameter, how should one choose it?

How to Choose K



- *a priori* knowledge about application domain
  - There are two kinds of people in the world: k = 2
  - There are five different types of bacteria: k = 5
- Search for a good k
  - Try different values of k and evaluate quality of results

Mitigating Dependence on Initial Centroids

Try multiple sets of randomly chosen initial centroids

Select “best” result, e.g.:

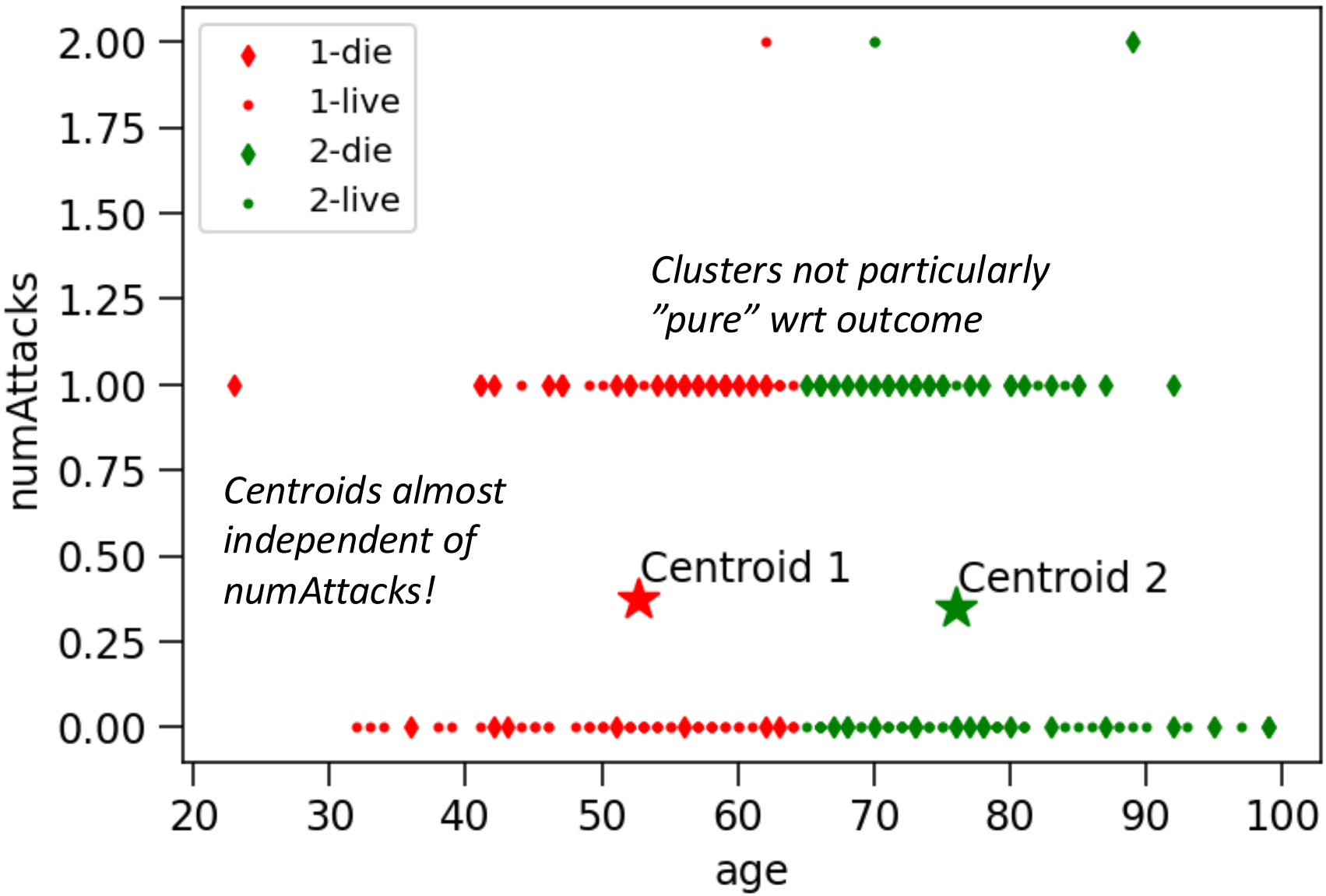
```
best = kMeans(points)
for t in range(num_trials):
   c = k_means(points)
   if total_inertia(c) < total_inertia(best):
      best = c
return best
```

Five-minute Break

Examine Results on Cardiac Data

- Let's try clustering with two just attributes and visualize what happens
- Before we saw that age and numAttacks looked like good variables to try

Kmeans, k=2, Using Age, NumAttacks



Variables Have Different Scales

- Age contributes much more to distance than numAttacks, since ages are much larger than numAttacks
- E.g., If centroid is age = 40, numAttacks = 0, then a person with age = 43, numAttacks = 0 further away than a person with age = 40, numAttacks = 2
- Solution: normalize attributes

Z-scaling

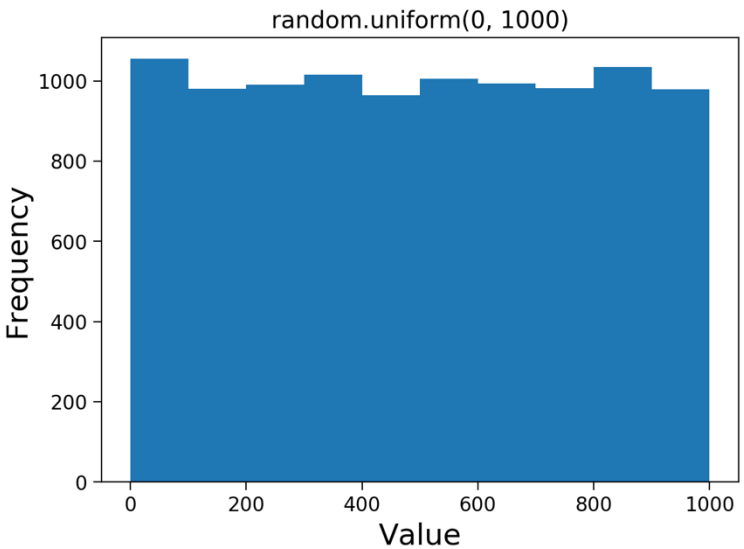
```
def scaleData(vals):
    #assumes vals a sequence of numbers
    vals = np.array(vals)
    mean = sum(vals)/len(vals)
    sd = np.std(vals)
    vals = vals - mean
    return vals/sd
```

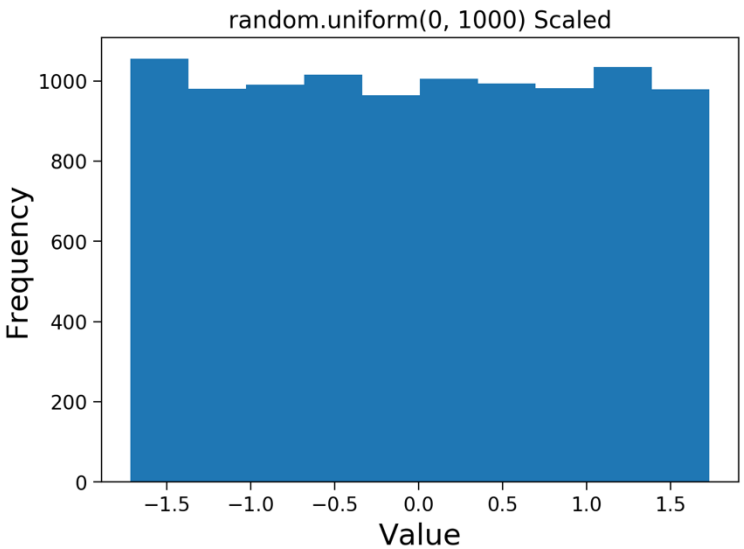
1) Changes values of data, but does not change “shape” of data distribution

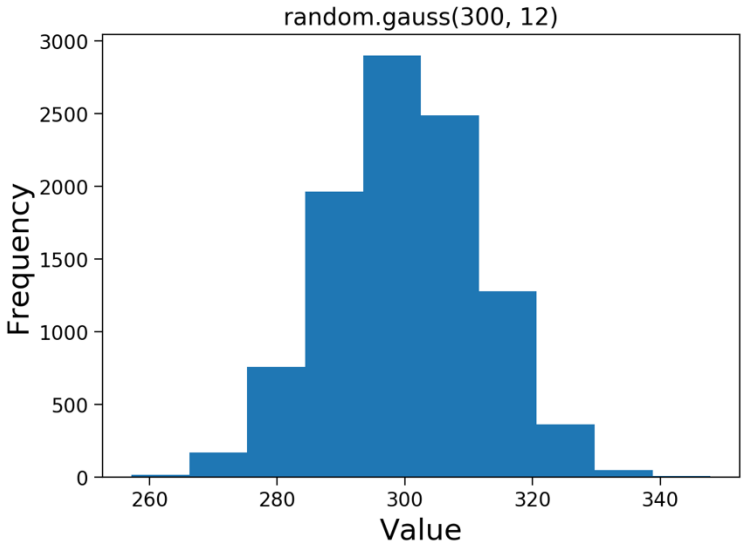
2) Changes mean of data to 0

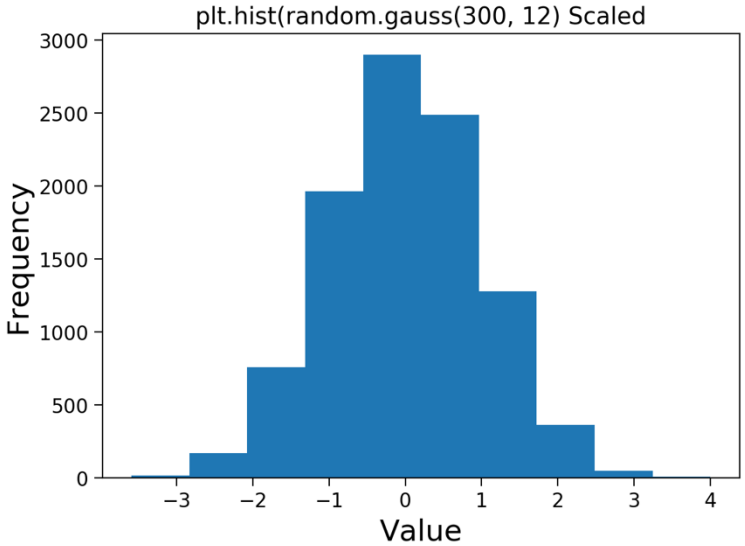
3) Changes standard deviation of data to 1

Scaling Changes Values But Not Shapes

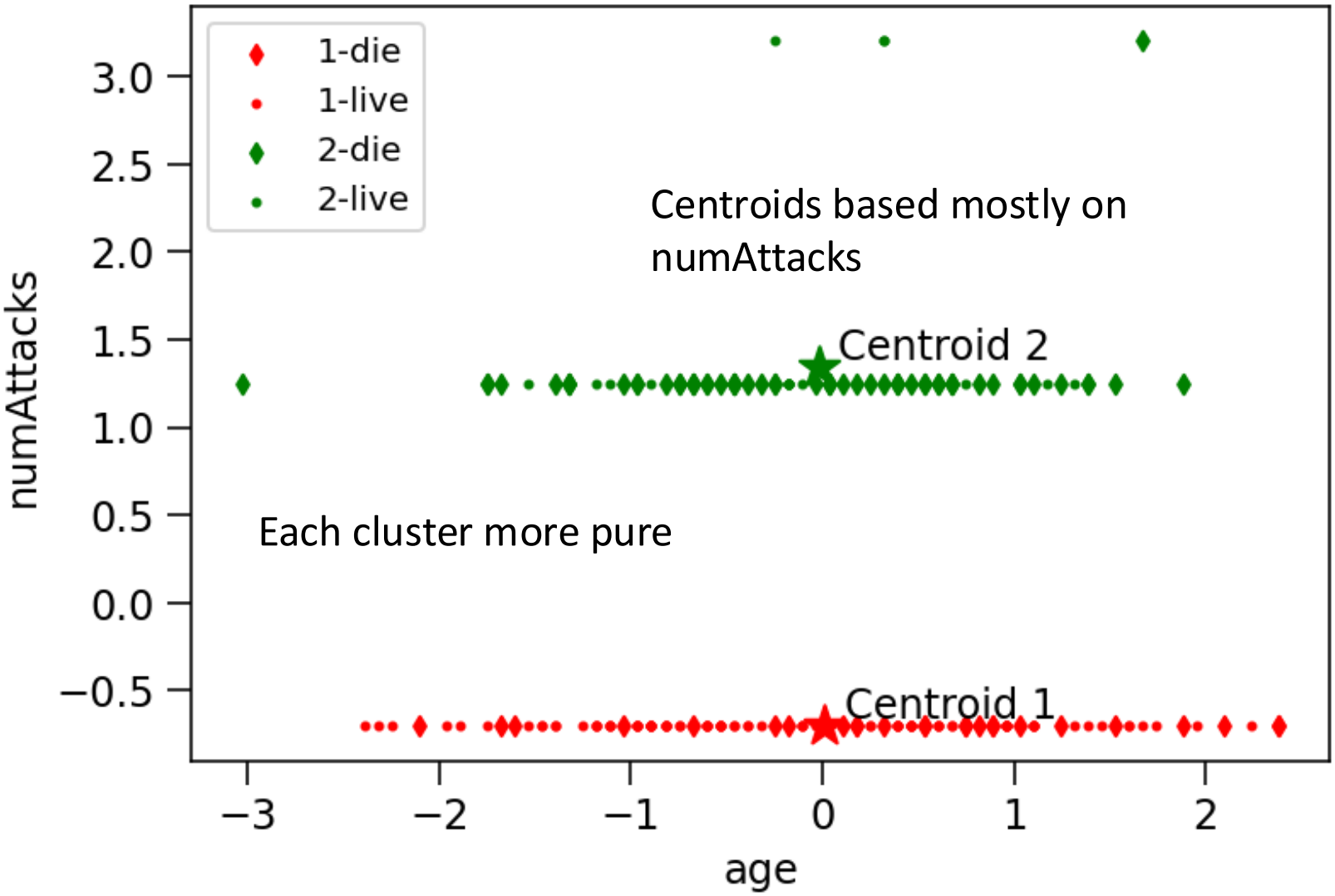








Kmeans w/ Scaling, k=2, Using Age, NumAttacks



Scaling Can Also Help with Logistic Regression

LR Results

Mean accuracy on test = 83%

Mean recall on test = 85%

Mean precision on test = 70%

LR Results with Scaling

Mean accuracy on test = 84%

Mean recall on test = 87%

Mean precision on test = 71%

Difference often much greater than for this example

Mean coefficients: HR = 0.002, stElev = 1.807, Age = 0.011, prevAttacks = 2.196

Let’s try Clustering Using all features

Test k-means (k = 2)

Cluster of size 224, frac. pos. = 0.290, num. pos. = 65

Cluster of size 26, frac. pos. = 0.692, num. pos. = 18

Using It as a Classifier

Label each cluster based on whether it has more positive or negatives

Test k-means (k = 2)
Cluster of size 224, frac. pos. = 0.290, num. pos. = 65, Guess = Negative
Cluster of size 26, frac. pos. = 0.692, num. pos. = 18, Guess = Positive

Accuracy = 71% 177/250
Sensitivity = 22% 18/(18+65)
Specificity = 95% 159/(159+8)

LR Results
Mean accuracy on test = 83%
Mean recall on test = 85%
Mean precision on test = 70%

```
                Pred = 0     Pred = 1
                 ----------------------
Actual = 0   | 159          8
Actual = 1   | 65           18
```

A Hypothesis

- Different subgroups of positive patients have different characteristics
- How might we test this?
- Try some other values of k

Testing Multiple Values of k

Test k-means (k = 2)

Overall Total correct = 177 out of 250 (0.708)
Sensitivity = 0.217
Specificity = 0.952

Test k-means (k = 4)

Overall Total correct = 209 out of 250 (0.836)
Sensitivity = 0.867
Specificity = 0.820

Test k-means (k = 6)

Overall Total correct = 209 out of 250 (0.836)
Sensitivity = 0.867
Specificity = 0.820

Test k-means (k = 50)

Overall Total correct = 221 out of 250 (0.884)
Sensitivity = 0.831
Specificity = 0.910

What's going on??

10-fold cross validation (70/30 split)

k=2, accuracy = 0.747, 95% conf = [0.715, 0.779]
k=2, mean recall = 0.392, 95% conf = [0.195, 0.564]
k=2, mean precision = 0.918, 95% conf = [0.883, 0.951]

k=4, accuracy = 0.842, 95% conf = [0.824, 0.857]
k=4, mean recall = 0.850, 95% conf = [0.812, 0.891]
k=4, mean precision = 0.837, 95% conf = [0.823, 0.855]

k=6, accuracy = 0.841, 95% conf = [0.827, 0.859]
k=6, mean recall = 0.844, 95% conf = [0.804, 0.886]
k=6, mean precision = 0.837, 95% conf = [0.816, 0.851]

k=10, accuracy = 0.841, 95% conf = [0.825, 0.857]
k=10, mean recall = 0.781, 95% conf = [0.738, 0.819]
k=10, mean precision = 0.869, 95% conf = [0.853, 0.883]

k=20, accuracy = 0.840, 95% conf = [0.814, 0.858]
k=20, mean recall = 0.709, 95% conf = [0.657, 0.766]
k=20, mean precision = 0.901, 95% conf = [0.880, 0.919]

k=50, accuracy = 0.825, 95% conf = [0.805, 0.848]
k=50, mean recall = 0.676, 95% conf = [0.606, 0.731]
k=50, mean precision = 0.896, 95% conf = [0.876, 0.914]

10-fold cross validation

k=2, accuracy = 0.747, 95% conf = [0.715, 0.779]
k=2, mean recall = 0.392, 95% conf = [0.195, 0.564]
k=2, mean precision = 0.918, 95% conf = [0.883, 0.951]

k=4, accuracy = 0.842, 95% conf = [0.824, 0.857]
k=4, mean recall = 0.850, 95% conf = [0.812, 0.891]
k=4, mean precision = 0.837, 95% conf = [0.823, 0.855]

k=6, accuracy = 0.841, 95% conf = [0.827, 0.859]
k=6, mean recall = 0.844, 95% conf = [0.804, 0.886]
k=6, mean precision = 0.837, 95% conf = [0.816, 0.851]

k=10, accuracy = 0.841, 95% conf = [0.825, 0.857]
k=10, mean recall = 0.781, 95% conf = [0.738, 0.819]
k=10, mean precision = 0.869, 95% conf = [0.853, 0.883]

k=20, accuracy = 0.840, 95% conf = [0.814, 0.858]
k=20, mean recall = 0.709, 95% conf = [0.657, 0.766]
k=20, mean precision = 0.901, 95% conf = [0.880, 0.919]

k=50, accuracy = 0.825, 95% conf = [0.805, 0.848]
k=50, mean recall = 0.676, 95% conf = [0.606, 0.731]
k=50, mean precision = 0.896, 95% conf = [0.876, 0.914]

k=4 is best performing model

Larger k isn't a lot worse, but we should favor best performing, simplest model

Clustering not usually used for classification

Used it here for illustrative purposes

Summary

- ML is a different way of thinking about programming:
  - Use data to train a model
  - Use model to make predictions
- Unsupervised & supervised methods
  - Unsupervised methods, e.g., *clustering* don't require labeled data to learn
    - In some cases, e.g., with *patients*, clusters correspond to outcomes
  - Supervised methods use labeled data to build models tht predict outcome
- Use a separate test data set to check for overfitting!

Parting Words About 6.100B

Four Ways to Study the World

- Theory
  - Mathematical thinking and skills
- Physical experiments
  - Experimental design and skills
- Computational experiments
  - Computational thinking and skills
- Data-intensive
  - Computer science and statistics

6.100B Themes

- Computational experiments
  - Modeling
  - Algorithmic thinking
  - Understanding when to trust results
  - Presenting and evaluating results
- Extracting information from data
  - Visualizing data
  - Manipulating data
  - Statistical analysis
  - Machine learning

6.100B Major Topics

- Optimization problems
- Stochastic thinking
- Modeling aspects of the world
- Becoming a better programmer
  - Exposure to a few extra features of Python and some useful libraries
  - Practice, practice, practice

Optimization Problems

- Many problems can be formulated in terms of
  - Objective function
  - Set of constraints
- Greedy algorithms often useful
  - But may not find optimal solution
- Many optimization problems inherently exponential
  - But dynamic programming often works
  - And memoization a generally useful technique
- Examples: knapsack problems, graph problems, curve fitting, clustering

Stochastic Thinking

- The world is (predictably) non-deterministic
- Thinking in terms of probabilities is often useful
- Randomness is a powerful tool for building computations that model the world
- Random computations useful even for problems that do not involve randomness
  - E.g., integration

Modeling the World

- Models always inaccurate: provide abstractions of reality
- Deterministic models, e.g., graph theoretic
- Statistical models
  - Simulation models: Monte Carlo simulation
  - Models based on sampling
    - Characterizing performance is critical
      - Central limit theorem, empirical rule, p-values
  - Machine learning
    - Classification, regression, clustering
- Making sense of data
  - Plotting
  - Good and bad practices

What's Next for You?

- Will AI make it unnecessary for humans to understand computing and computation?

"Trying to predict the future is like trying to drive down a country road at night with no lights while looking out the back window. " – Peter Drucker

Predicting the Future of Computing

- "I think there is a world market for maybe five computers." - Thomas Watson, chairman of IBM, 1943

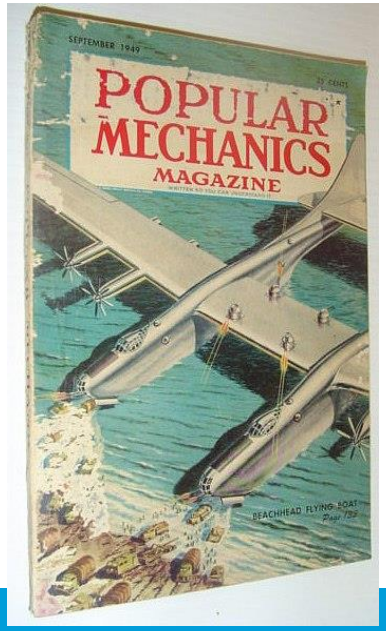
False

The "MAN" Makes the Business

By THOMAS J. WATSON

- "Computers in the future may weigh no more than 1.5 tons." - *Popular Mechanics*, forecasting the relentless march of science, 1949

True, but …

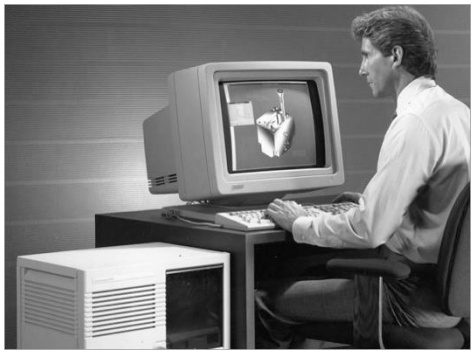


Predicting the Future of Computing

- "I have traveled the length and breadth of this country and talked with the best people, and I can assure you that data processing is a fad that won't last out the year." - The editor in charge of business books for Prentice Hall, 1957

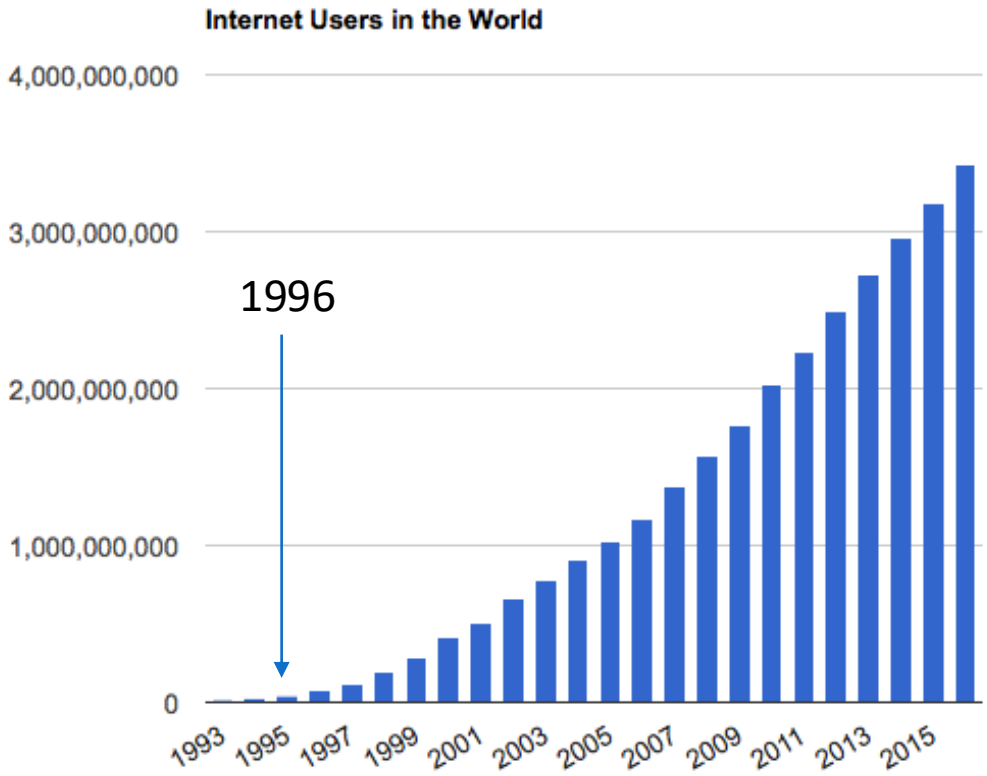
- "There is no reason anyone would want a computer in their home." - Ken Olson (MIT BS 1950, MS 1952), president, chairman & founder of Digital Equipment Corp. (once 2nd largest computer company in the world), 1977

PDP-4 (1963) and PDP-7 (1964)



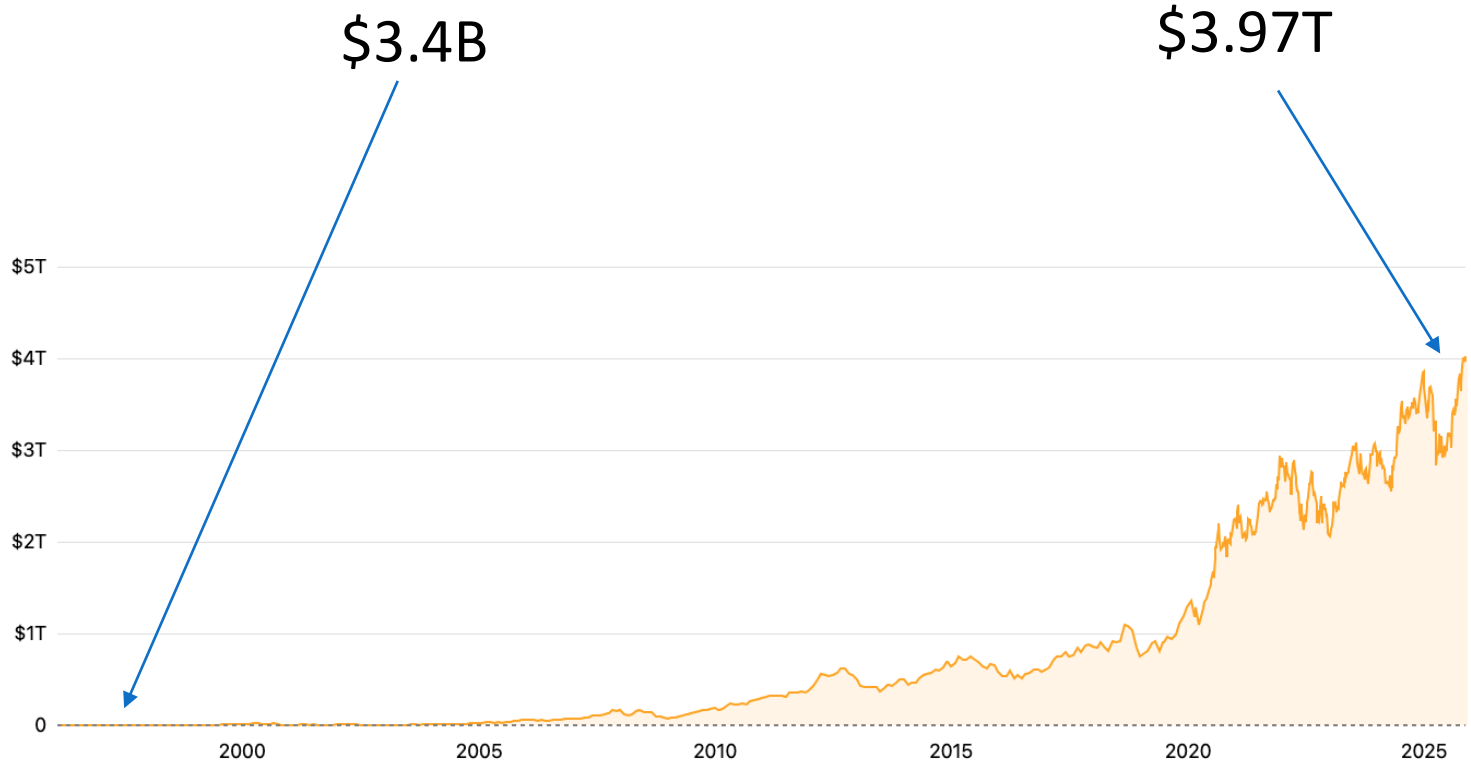
Predicting the Future of Computing

- "I predict the Internet will soon go spectacularly supernova and in 1996 catastrophically collapse." –Bob Metcalfe (MIT SB 1969), co-inventor of ethernet, Founder of 3Com



Predicting the Future of Computing

- "Apple is already dead." –Nathan Myhrvold, Microsoft CTO, 1997



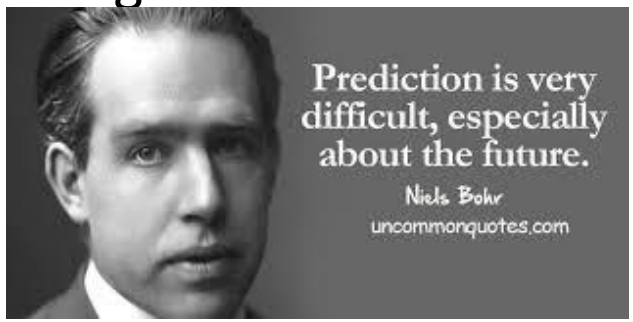
Apple's Market Cap

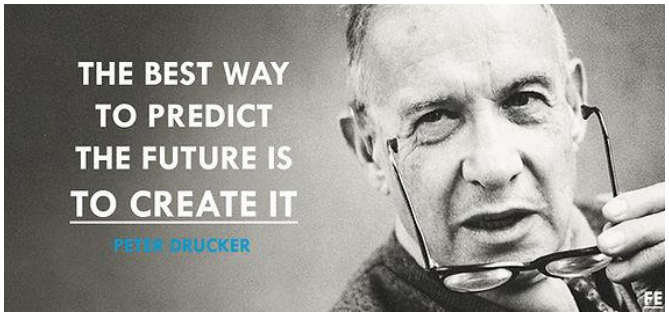
Predicting the Future of Computing

- "Two years from now, spam will be solved."—Bill Gates, Founder of Microsoft, 2004

A Brief History of Computing, One Man's Perspective

- 1960's building programs that work at all
- 1970's time sharing/mainframes
- 1980's personal computing
- 1990's Internet/World Wide Web
- 2000's Cloud computing
- 2010's Mobile computing
- 2020's AI/doing more with more
- 2030's





- 2030's Human-AI symbiosis, doing more with less?

So, Should Students Study Computation?

I have been asked, “why should students learn about computation when generative AI can write code for them?”

*A*s Charles Babbage said, *"I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question.*

Computational thinking and programming are not the same thing. Programming is a skill. Computational thinking is a mode of thought.

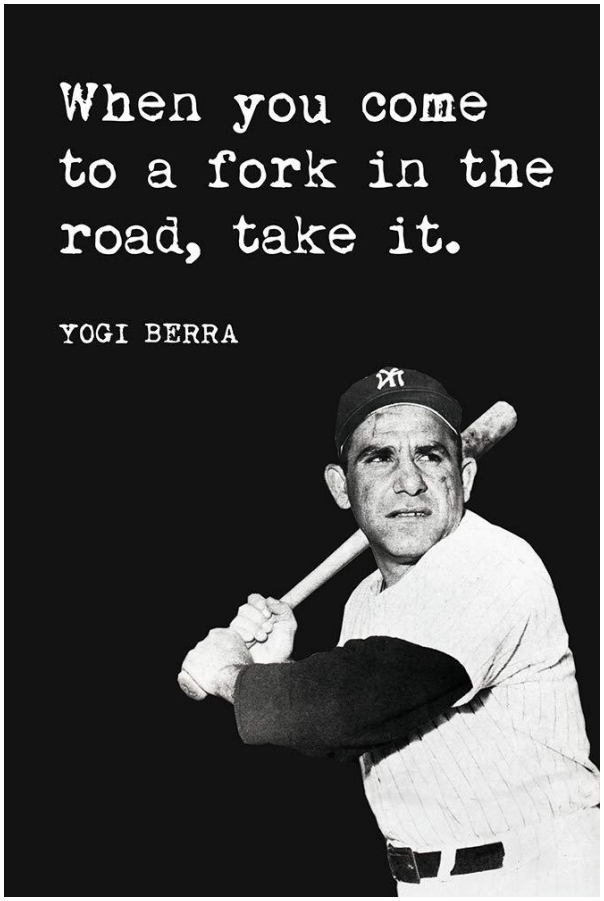
So, Should Students Study Computation?

From purely utilitarian perspective, the demand for those capable of writing only mundane code will surely decline.

But the demand for those who can **thoughtfully** harness the power of computation to solve previously unsolved problems will probably rise.

And an MIT education should not be narrowly utilitarian. Why do we have humanities requirements? To provide students with new ways to appreciate the world, and to stimulate their minds. Computation has become central to society. Understanding it is an important part of a well-rounded education.

Thank you for taking 6.100B

Good luck with the final quiz

And always remember